CS 5402: Introduction to Data Mining

SP2026

(Lecture - 15)

Neural Networks (NN)

Dr. Huiyuan Yang

Department of Computer Science
Missouri S&T

https://hyang428.github.io

MISSOURI S&T

Neural Networks Objectives

You should be able to...

- Explain the biological motivations for a neural network
- Combine simpler models (e.g. linear regression, binary logistic regression, multinomial logistic regression) as components to build up feed-forward neural network architectures
- Explain the reasons why a neural network can model nonlinear decision boundaries for classification
- Identify (some of) the options available when designing the architecture of a neural network
- Implement a feed-forward neural network

Agenda

- History of Neural Network
- Multilayer Neural Network
- Forward propagation
- Backward propagation
- Optimization Algorithms

History of Neural Networks

- Foundation of neural networks: 1943
- Perceptron: 1957
- < The AI Winter>
- Backpropagation:1986 (SVM, RF, GBDT: 1995-2009)
- The Rise of Deep Learning: Deep NN: 2006 Hinton
- 2012: golden ages of AI
  - Lots of data
  - Lots of computational resources
  - New algorithms

Inspiration: The Brain

- Many machine learning methods inspired by biology, e.g., the (human) brain
- Our brain has $\sim 10^{11}$ neurons, each of which communicates (is connected) to $\sim 10^{4}$ other neurons

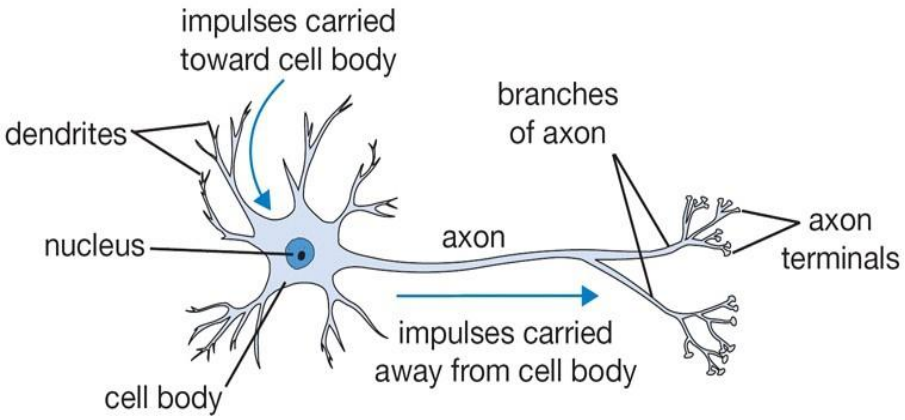


Figure : The basic computational unit of the brain: Neuron

credit: [http://cs231n.github.io/neural-networks-1/]

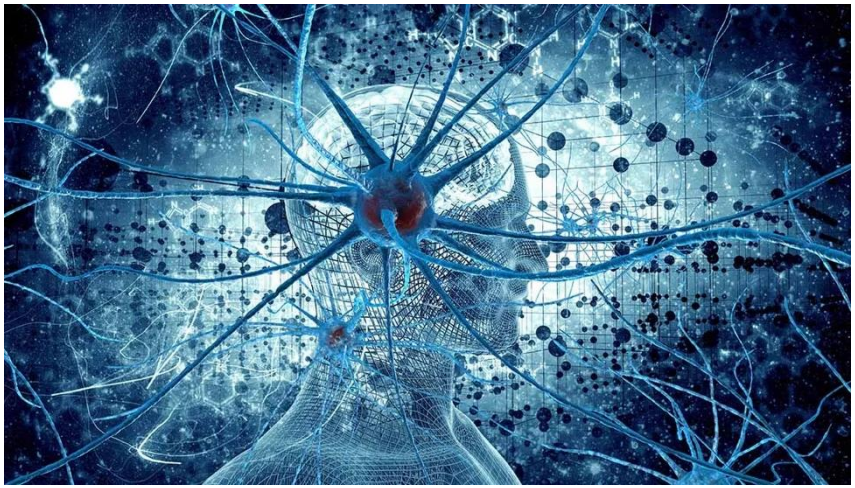
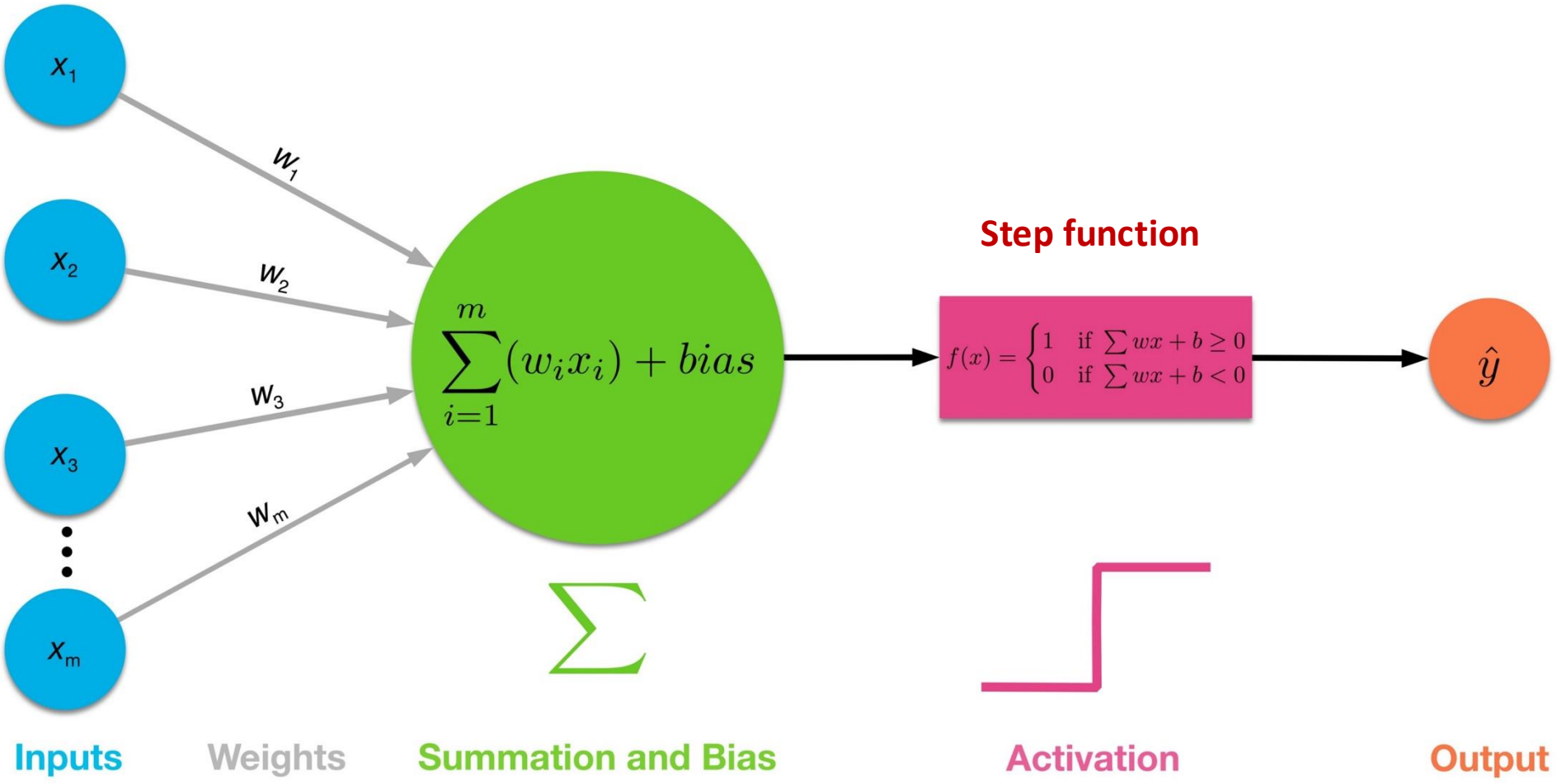
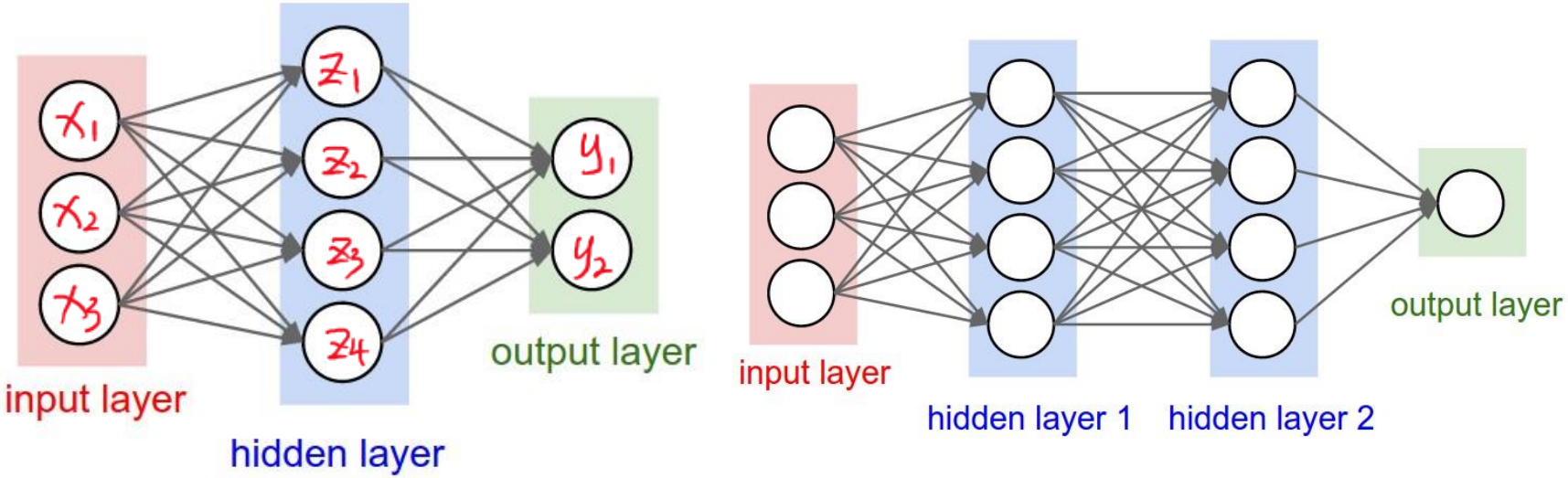


Image credit to: https://vitalrecord.tamhsc.edu/synapses-crucial-connections/

Perceptron (1957)



Multilayer perceptron



$$\begin{cases} z_1 = w_1^1 x + b_1^1 \\ z_2 = w_2^1 x + b_2^1 \\ \vdots \end{cases}$$

$$\begin{cases} y_1 = w_1^2 z + b_1^2 \\ y_2 = w_2^2 z + b_2^2 \end{cases}$$

$$y_1 = w_1^2 [z_1, z_2, z_3, z_4]^T + b_1^2$$

$$= w^* x + b^*$$

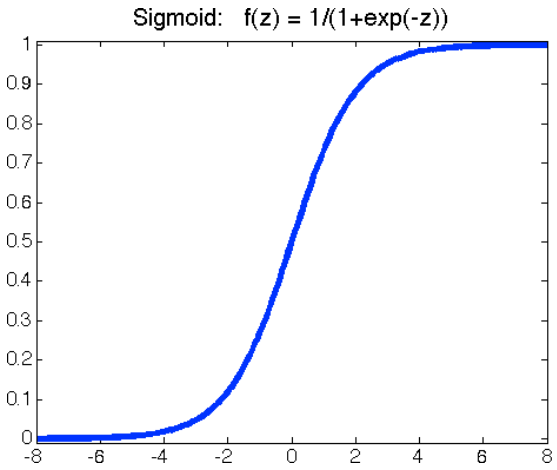
Linear function !

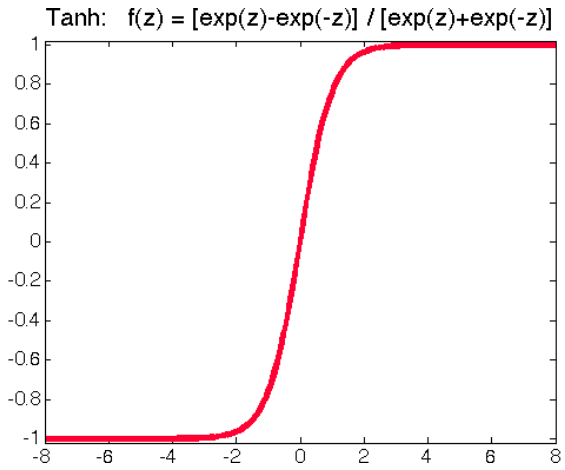
Activation Functions (non-linear function)

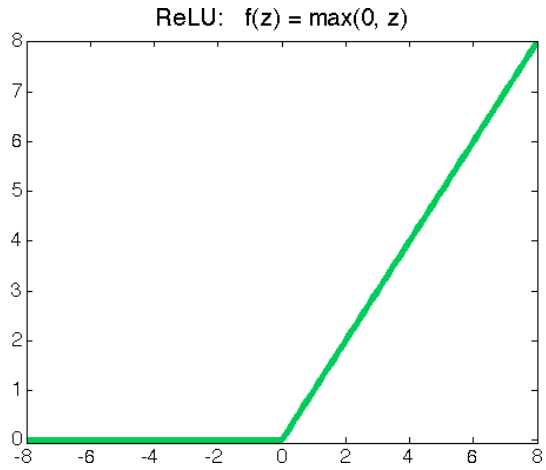
Most commonly used activation functions:

- Sigmoid: $\sigma(z) = \frac{1}{1+\exp(-z)}$
- Tanh: $\tanh(z) = \frac{\exp(z)-\exp(-z)}{\exp(z)+\exp(-z)}$
- ReLU (Rectified Linear Unit): $\mathrm{ReLU}(z) = \max(0, z)$

1980 and 90s: sigmoid & tanh



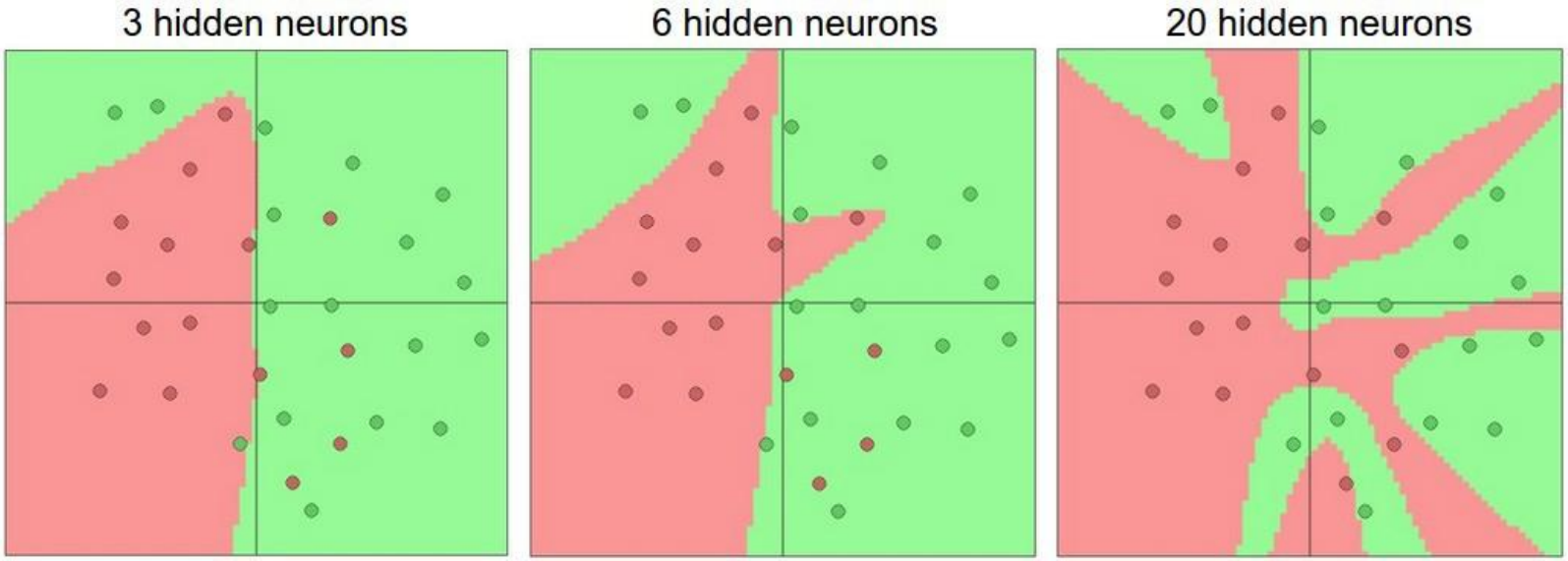




Universal Approximation

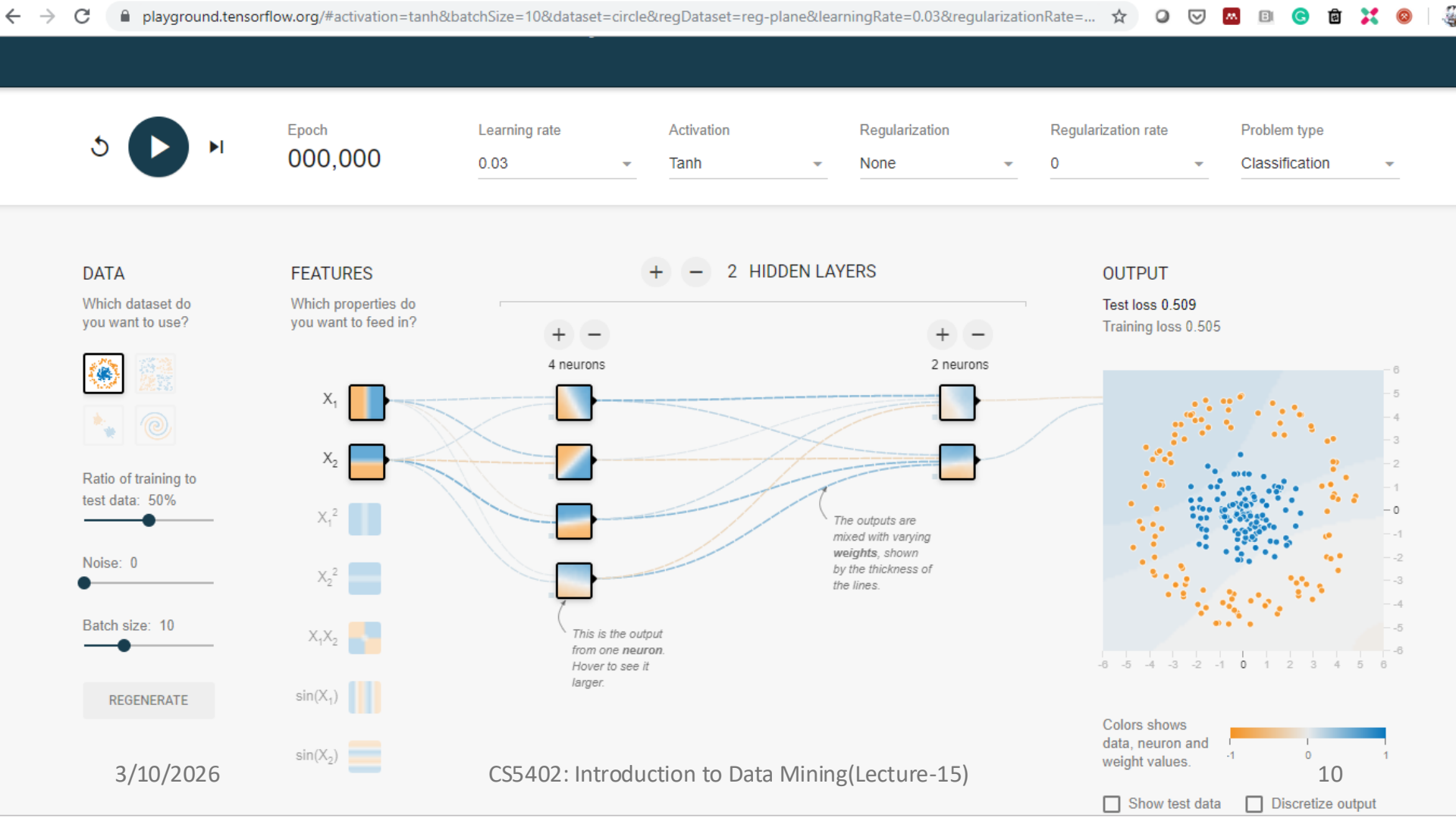
- Neural network with at **least one hidden layer** is a universal approximator (can represent any function) with arbitrary precision* .

  Proof in: Approximation by Superpositions of Sigmoidal Function, Cybenko, paper

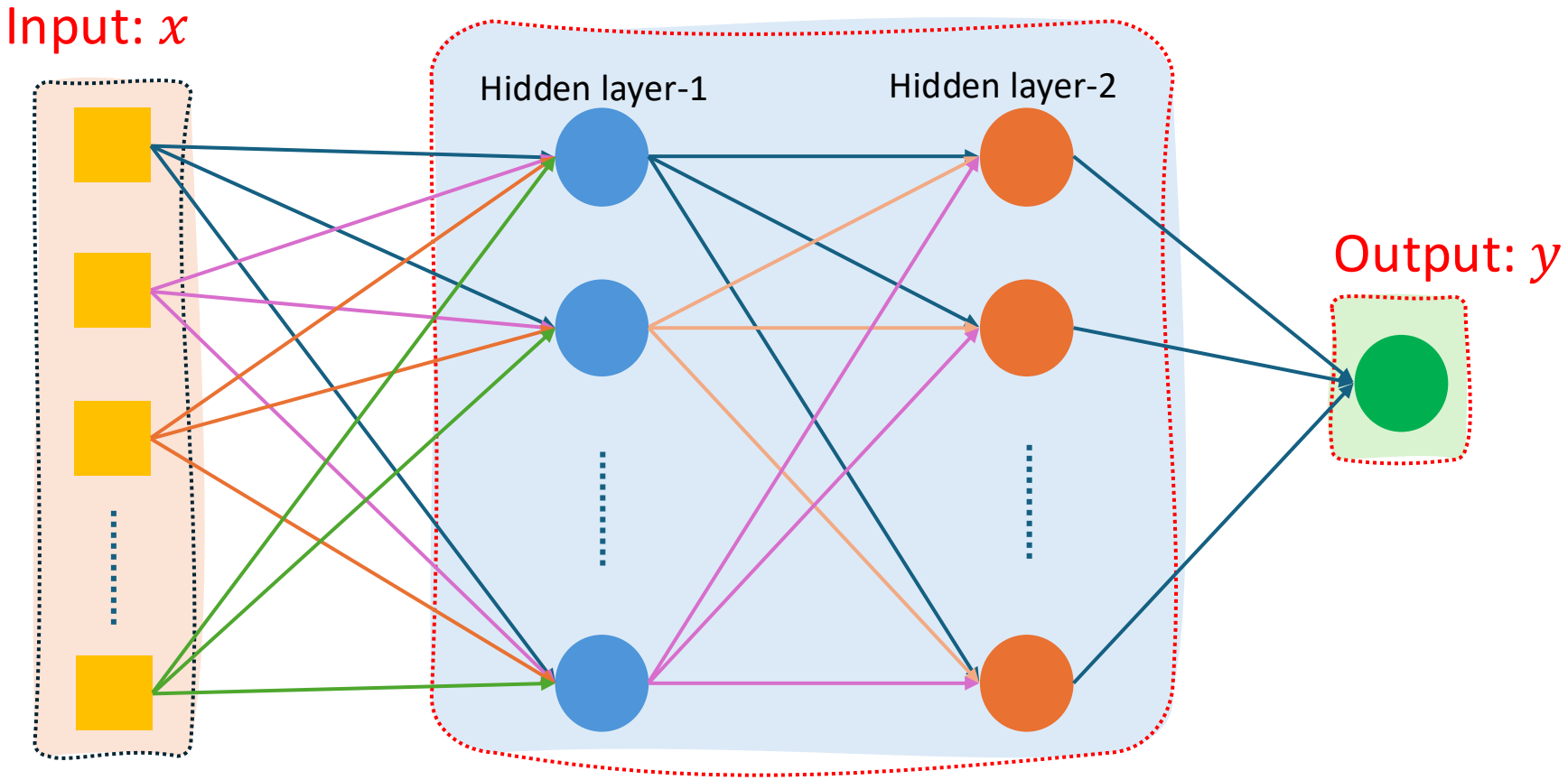


Fun Demo!!

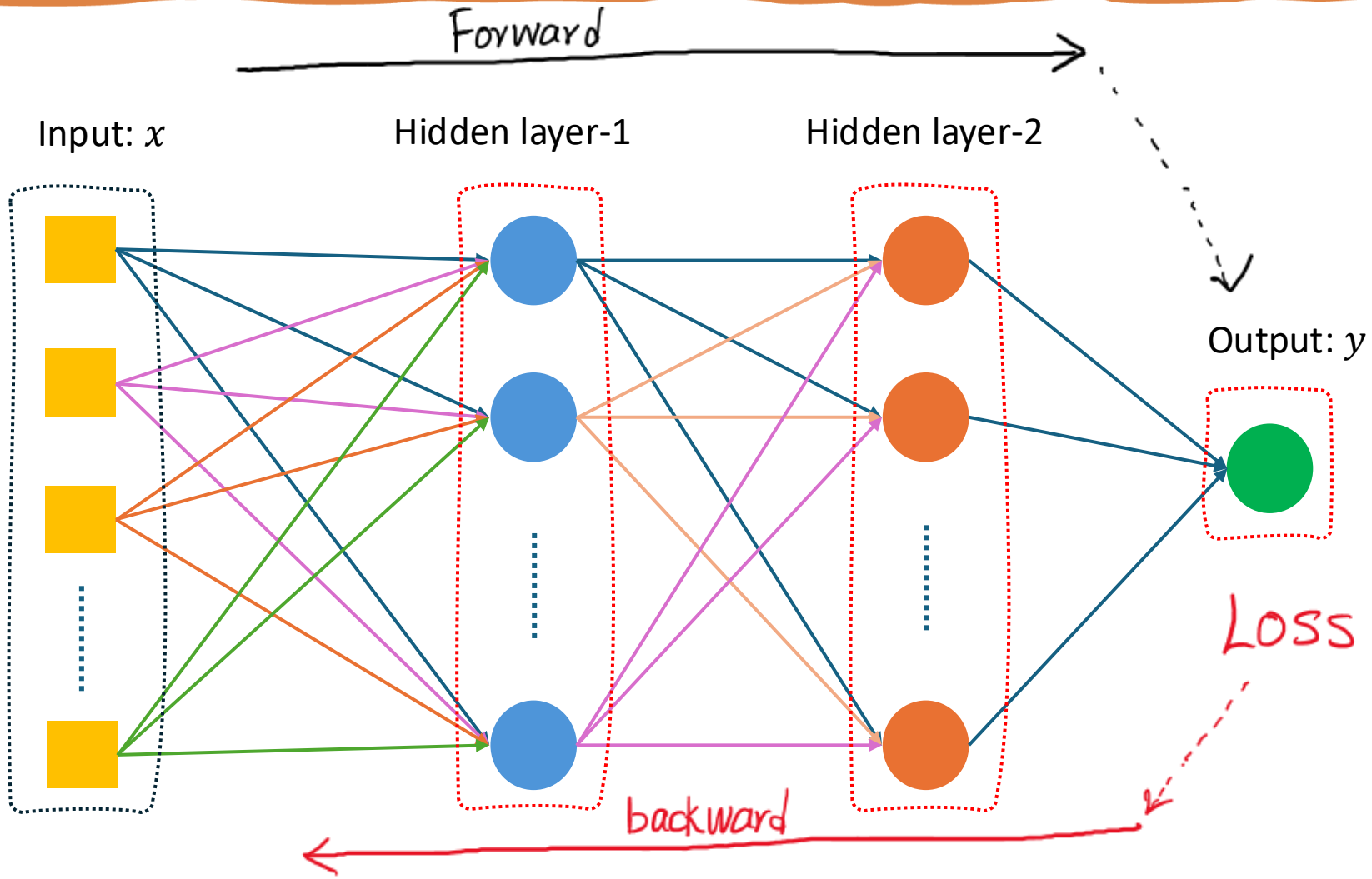
https://playground.tensorflow.org/#activation=tanh&batchSize=10&dataset=circle&regDatase



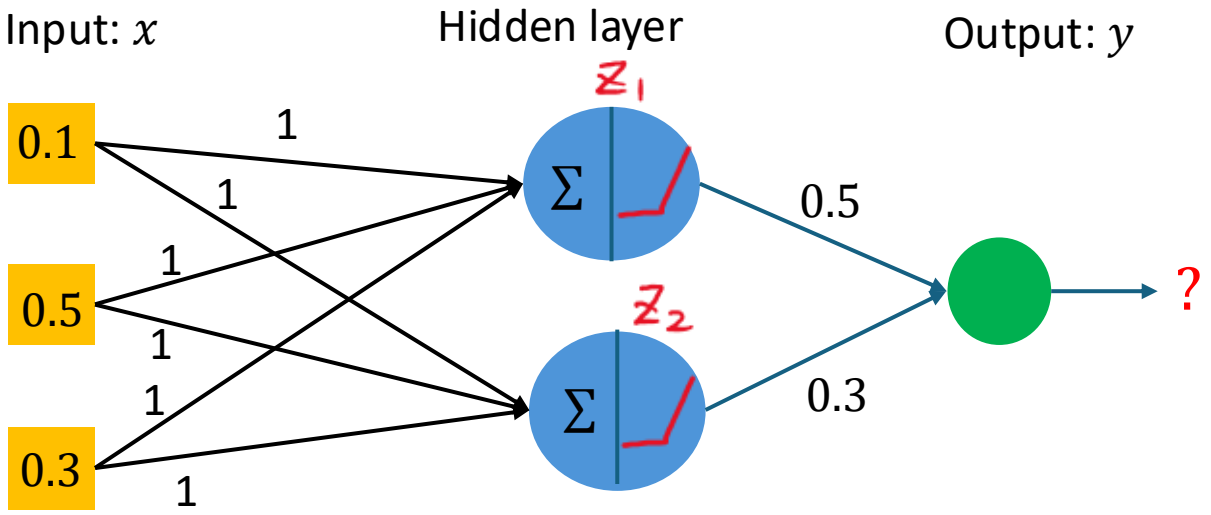
Key components



Two Operations (Forward, backward)



Example: Forward Propagation



$z_1 = \text{ReLU}(0.1 \times 1 + 0.5 \times 1 + 0.3 \times 1) = 0.9$

$z_2 = \text{ReLU}(0.1 \times 1 + 0.5 \times 1 + 0.3 \times 1) = 0.9$

$\hat{y} = z_1 \times 0.5 + z_2 \times 0.3 = 0.72$

Loss Function

Find the best weights using training data so that loss function is minimized (regression: squared error, for classification: cross entropy)

- Regression: $L = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y_i})^2$ MAE
- Clasification: $L = -\frac{1}{n}\sum_{i=1}^{n}\left(y_i \log(\hat{y_i}) + (1 - y_i)\log(1 - \widehat{y_i})\right)$

Cross-entropy loss

Loss Function (Cont'd)

binary classification.

- Clasification: $L = -\frac{1}{n}\sum_{i=1}^{n}\left(y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)\right)$

Cross-entropy loss

① Ground Truth $y=1$. $L = -y \cdot \log(\hat{y})$

$\Rightarrow \hat{y} = 0.9$. $L \approx 0.05$

$\Rightarrow \hat{y} = 0.1$ $L \approx 2.3$

$\rightarrow \hat{y} \rightarrow 1$

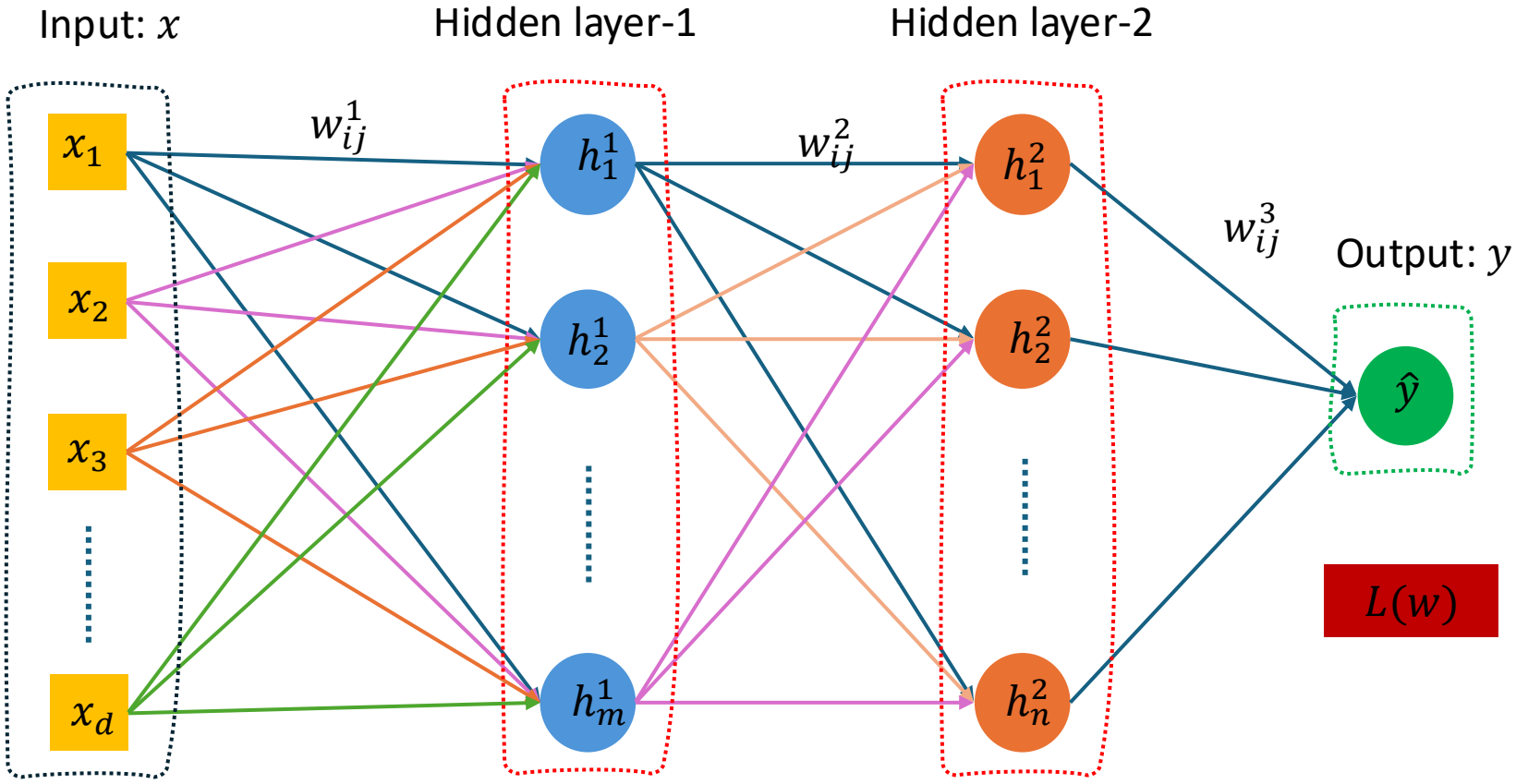
② Groud Truth $y=0$. $L = -1 \cdot \log(1-\hat{y})$

$\Rightarrow \hat{y} = 0.9$. $L \approx 2.3$

$\Rightarrow \hat{y} = 0.1$ $L \approx 0.05$

$\rightarrow \hat{y} \rightarrow 0$

Optimization Algorithm



Gradient Descent Algorithm (GD):

$$w^{k+1} = w^k - \eta \frac{\partial L}{\partial w}$$

Gradient Descent

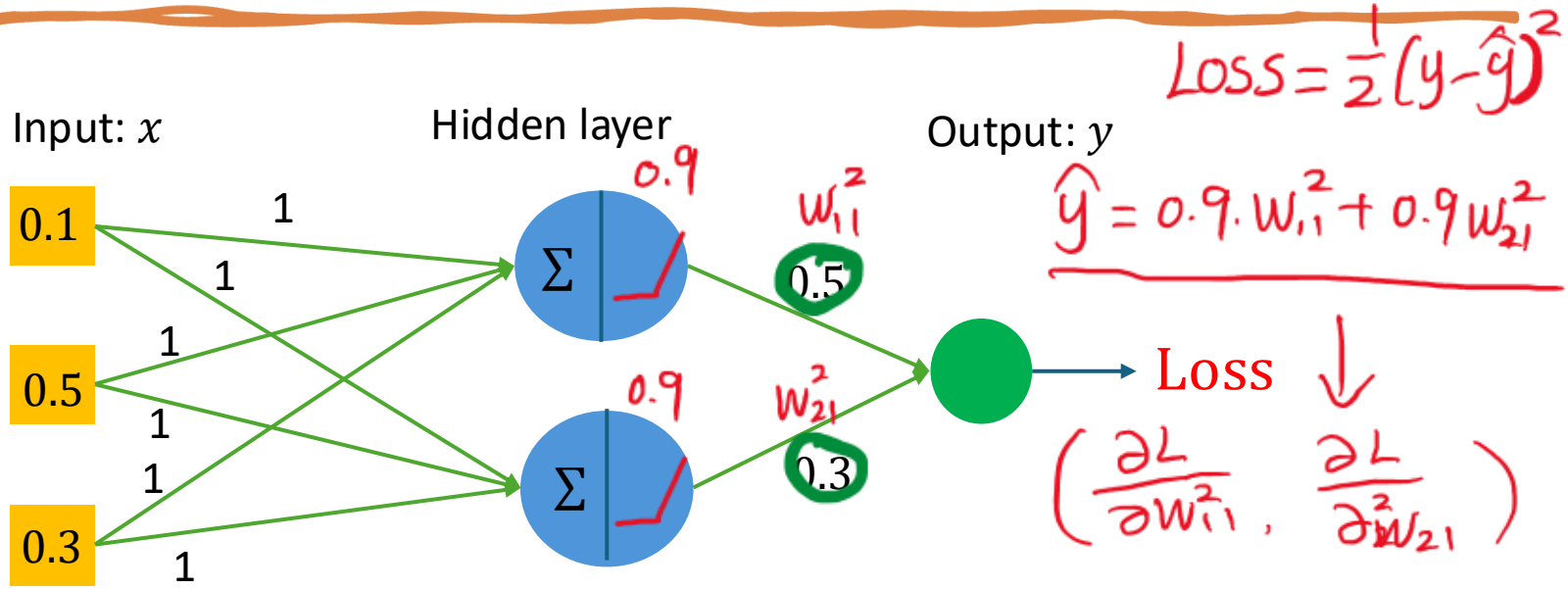
Gradient Descent Algorithm (GD):

$$w^{[k+1]} = w^{[k]} - \eta \frac{\partial L}{\partial w}$$

How to calculate $\frac{\partial L}{\partial w_{ij}^{l}}$

$h_\theta(x)$

Backward Propagation

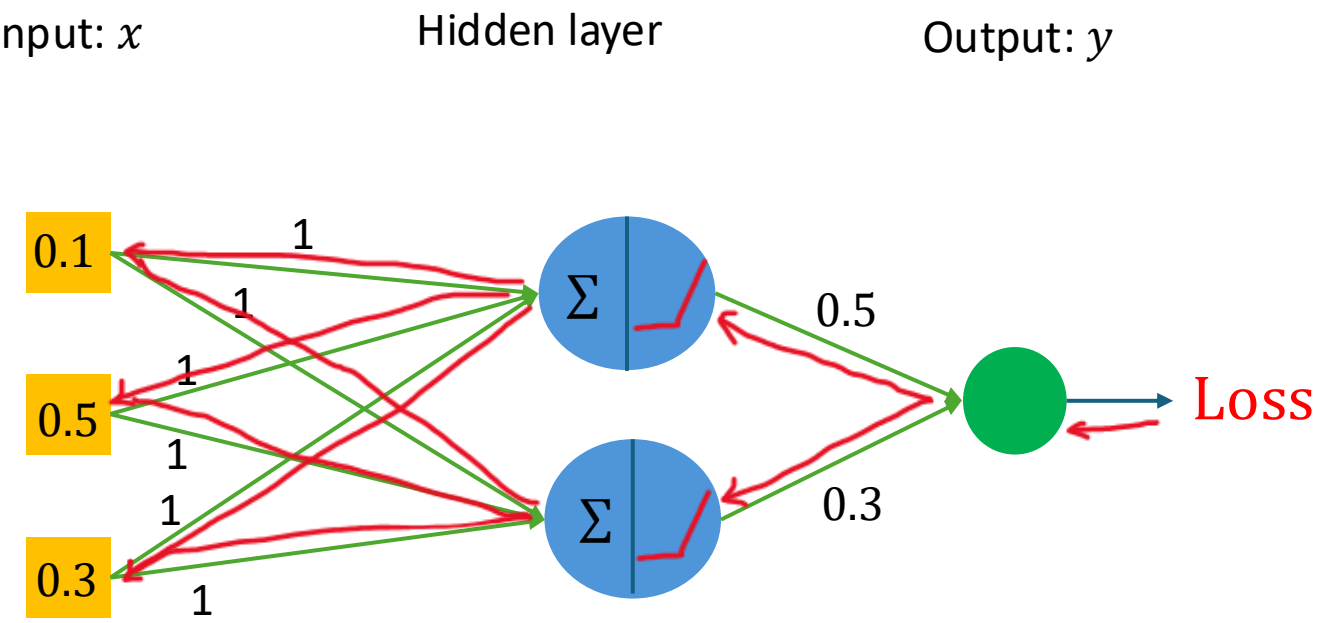


$z_1 = \text{ReLU}(0.1\times1 + 0.5\times1 + 0.3\times1) = 0.9$

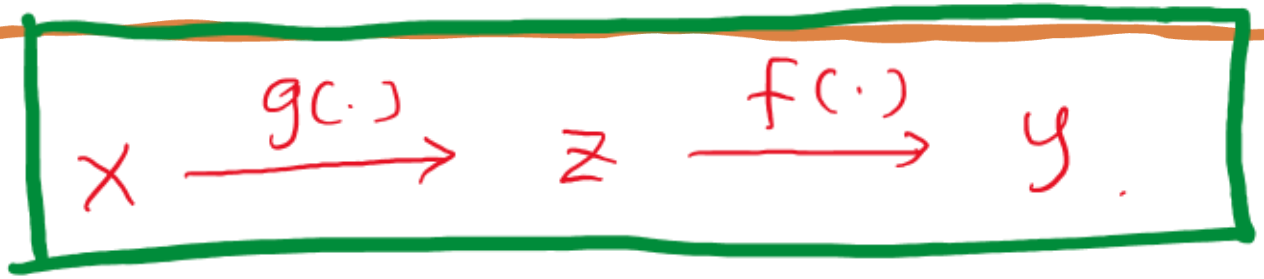
$z_2 = \text{ReLU}(0.1\times1 + 0.5\times1 + 0.3\times1) = 0.9$

$\hat{y} = z_1 \times 0.5 + z_2 \times 0.3 = 0.72$

Backward Propagation (Cont'd)



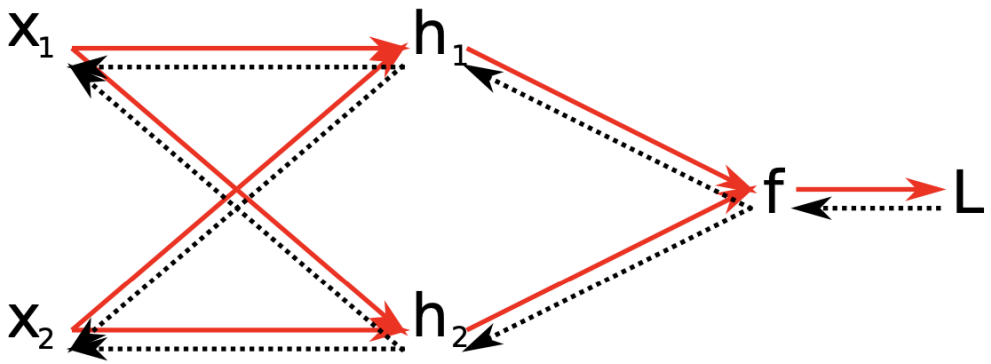
Chain Rule



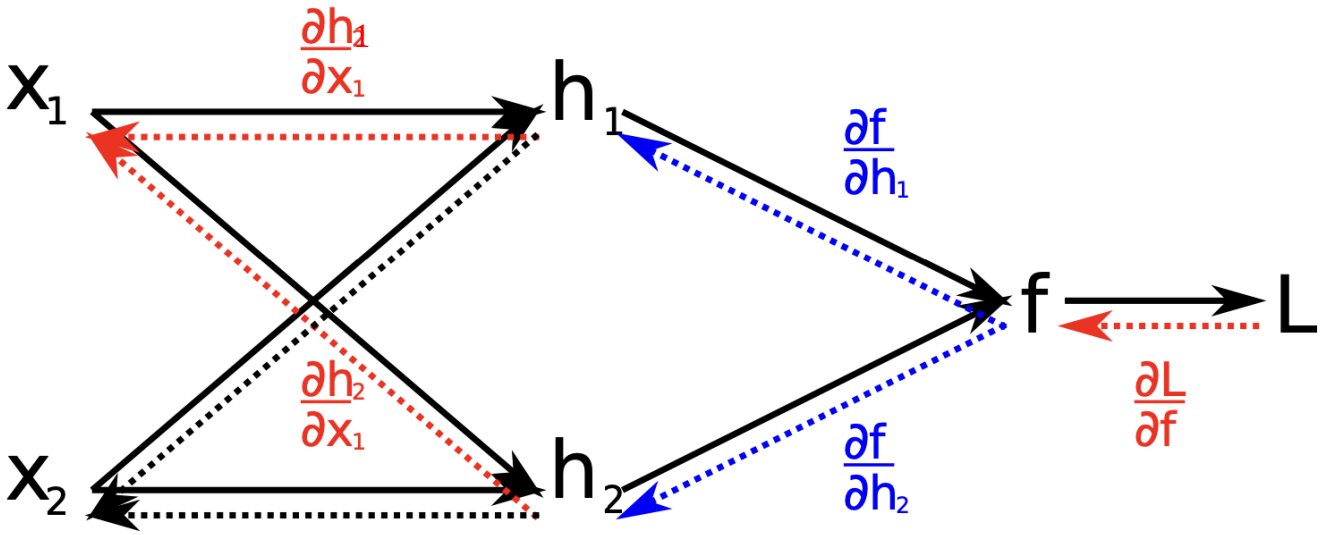
$$y = f(g(x))$$

$$\frac{dy}{dx} = f'(g(x)) \cdot g'(x) = \frac{dy}{dz} \cdot \frac{dz}{dx}$$

Chain Rule with multiple paths (1)



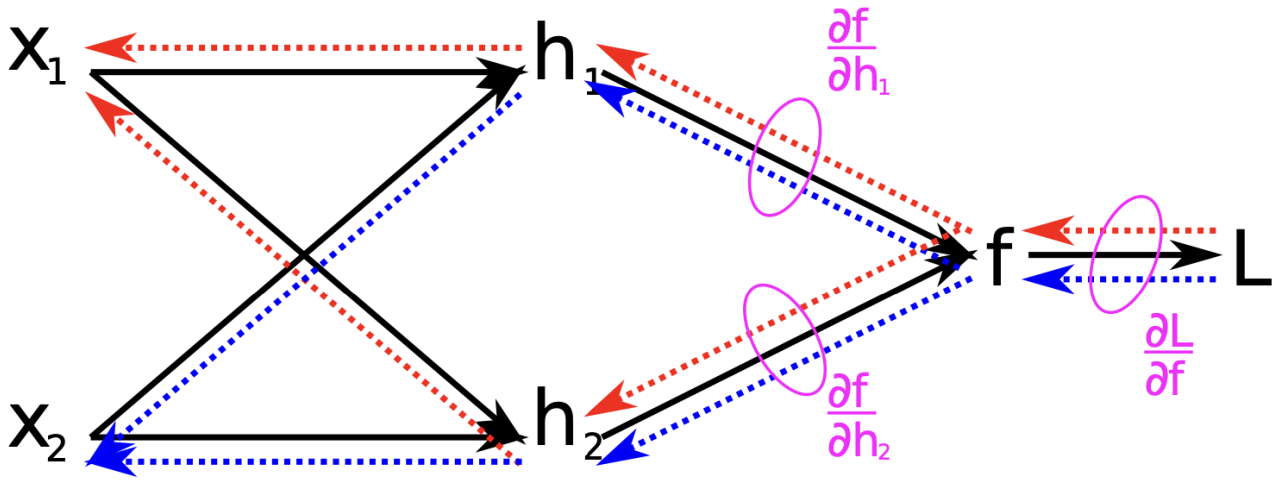
Chain Rule with multiple paths (2)



Chain rule of derivatives:

$$\frac{\partial L}{\partial x_1} = \frac{\partial L}{\partial f}\frac{\partial f}{\partial x_1} = \frac{\partial L}{\partial f}\left(\frac{\partial f}{\partial h_1}\frac{\partial h_1}{\partial x_1} + \frac{\partial f}{\partial h_2}\frac{\partial h_2}{\partial x_1}\right)$$

Chain Rule with multiple paths (3)



Local derivatives are *shared*:

$$\frac{\partial L}{\partial x_1} = \frac{\partial L}{\partial f}\left(\frac{\partial f}{\partial h_1}\frac{\partial h_1}{\partial x_1} + \frac{\partial f}{\partial h_2}\frac{\partial h_2}{\partial x_1}\right)$$

$$\frac{\partial L}{\partial x_2} = \frac{\partial L}{\partial f}\left(\frac{\partial f}{\partial h_1}\frac{\partial h_1}{\partial x_2} + \frac{\partial f}{\partial h_2}\frac{\partial h_2}{\partial x_2}\right)$$

Shared derivatives!

Memorization

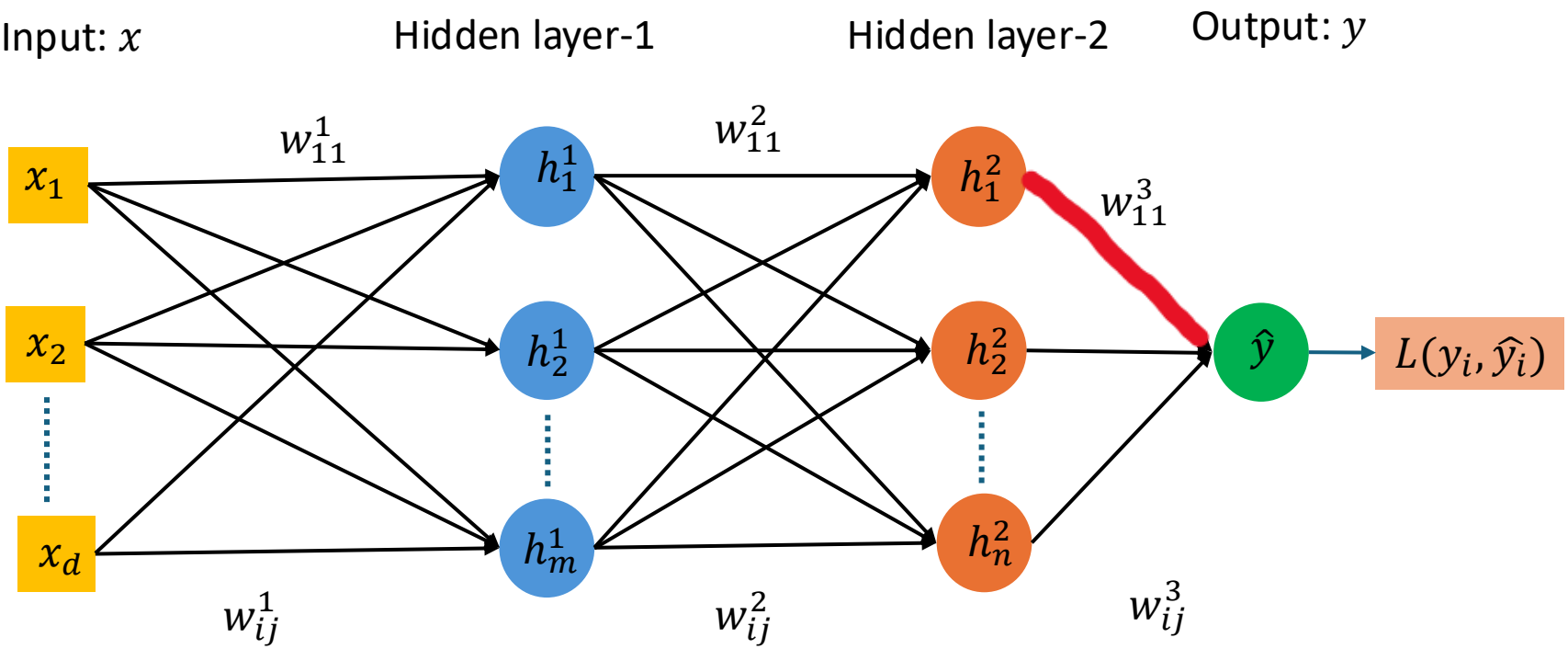
- To compute weights, we compute $\frac{\partial L}{\partial w_{ij}^{l}}$ so many times.
- If there is any operation that is done many times repeatedly, it is a good idea to compute it once and store it

=> Speed up

Backpropagation

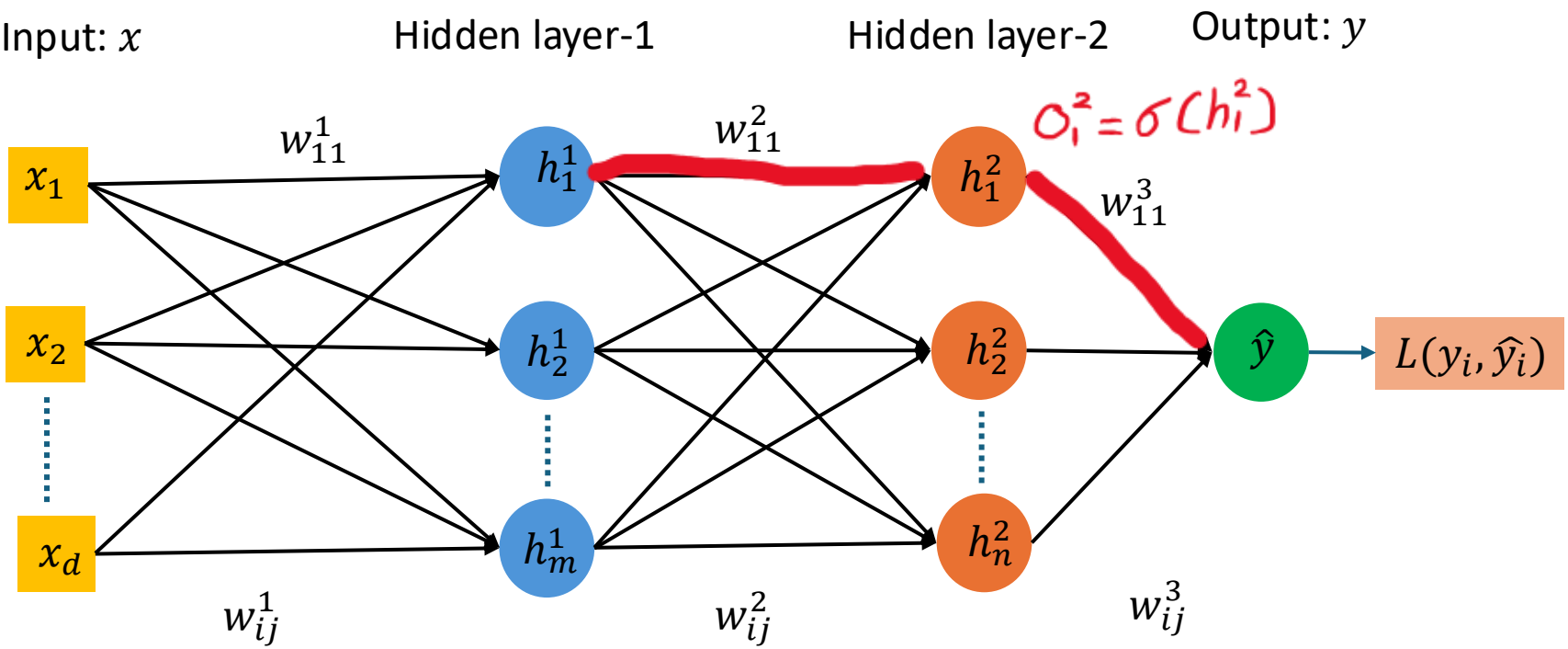
- Dataset: D={x_i, y_i}
- 1) Initialize w_{ij}^{l}
- 2) For each x_i in D
  - Pass x_i forward to the network (**forward propagation**)
  - Compute L(y_i, $\hat{y}_i$)
  - Compute all derivatives using chain rules and memorization
  - Update weights from end of the network to the start (**backpropagation)**
- 3) Repeat process 2 until convergence ($w_{ij-new} \sim= w_{ij-old}$)

Backward Propagation (1)



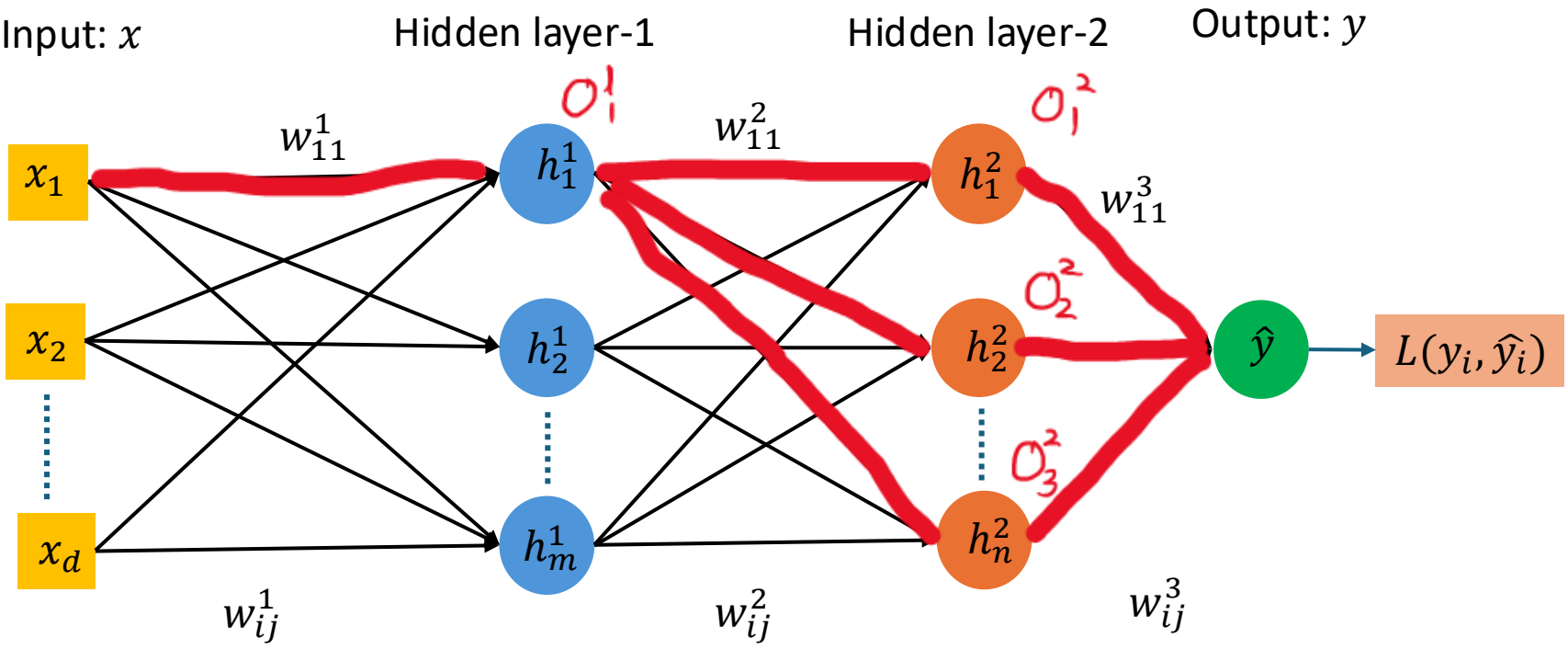
$$\frac{\partial L}{\partial w_{11}^3} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w_{11}^3}$$

Backward Propagation (2)



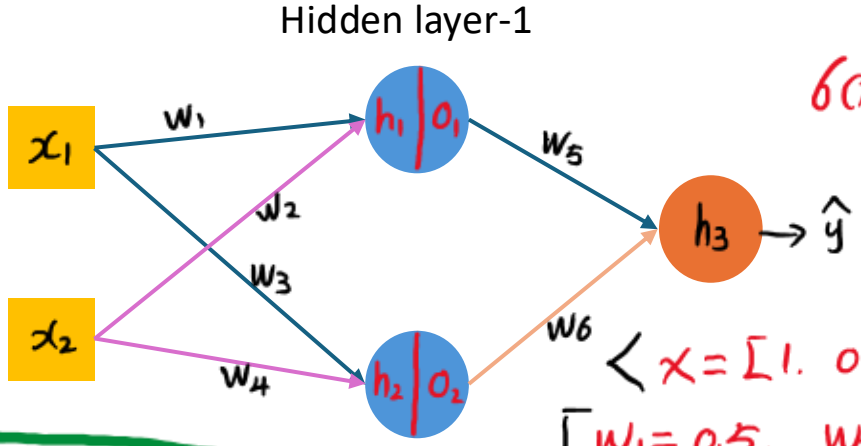
$$\frac{\partial L}{\partial w_{11}^2} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial O_1^2} \cdot \frac{\partial O_1^2}{\partial h_1^2} \cdot \frac{\partial h_1^2}{\partial w_{11}^2}$$

Backward Propagation (3)



$$\frac{\partial L}{\partial w_{11}^1} = \frac{\partial L}{\partial \hat{y}} \cdot \left(\frac{\partial \hat{y}}{\partial O_1^2} \cdot \frac{\partial O_1^2}{\partial h_1^2} \cdot \frac{\partial h_1^2}{\partial O_1^1} + \frac{\partial \hat{y}}{\partial O_2^2} \cdot \frac{\partial O_2^2}{\partial h_2^2} \cdot \frac{\partial h_2^2}{\partial O_1^1} + \cdots + \frac{\partial \hat{y}}{\partial O_n^2} \cdot \frac{\partial O_n^2}{\partial h_n^2} \cdot \frac{\partial h_n^2}{\partial O_1^1} \right) \cdot \frac{\partial O_1^1}{\partial h_1^1} \cdot \frac{\partial h_1^1}{\partial W_{11}^1}$$

Example (1)



$$\sigma(x) = \text{ReLU}(x) = \begin{cases} x & : \text{if } x \geq 0 \\ 0 & : \text{otherwise} \end{cases}$$

$$\hat{y} = h_3$$

$$h_3 = w_5 \cdot o_1 + w_6 \cdot o_2$$

$$o_1 = \sigma(h_1) = \sigma(w_1 \cdot x_1 + w_2 \cdot x_2)$$

$$o_2 = \sigma(h_2) = \sigma(w_3 \cdot x_1 + w_4 \cdot x_2)$$

$\langle x = [1.\ 0.5]^T, \quad y = 4 \rangle$

$$\begin{bmatrix} w_1 = 0.5 & w_2 = 1.5 & w_3 = 2.3 \\ w_4 = 3 & w_5 = 1 & w_6 = 1 \end{bmatrix}$$

$h_1 = w_1 x_1 + w_2 x_2 = 1.25 \Rightarrow o_1 = \sigma(h_1) = 1.25$

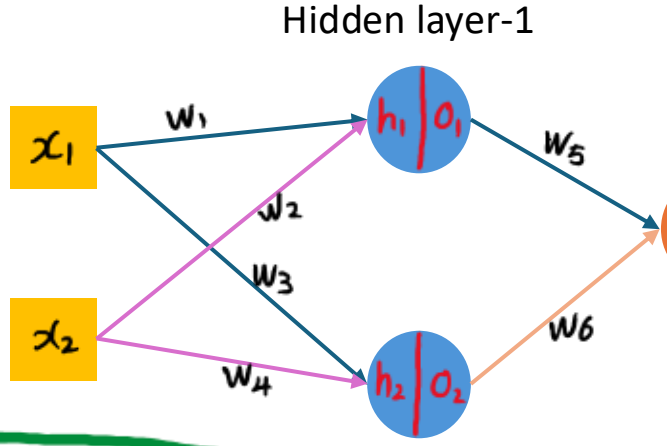
$h_2 = w_3 x_1 + w_4 x_2 = 3.8 \Rightarrow o_2 = \sigma(h_2) = 3.8$

$h_3 = w_5 \cdot o_1 + w_6 \cdot o_1 = 5.05$

$\hat{y} = h_3 = 5.05$

$$\text{Loss} = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2 = (4 - 5.05)^2 = 1.1025$$

Example (2)



$$\sigma(x) = \text{ReLU}(x) = \begin{cases} x & : \text{if } x \geq 0 \\ 0 & : \text{otherwise} \end{cases}$$

$\langle x = [1,\ 0.5]^T,\quad y = 4 \rangle$

$$\begin{bmatrix} w_1 = 0.5, & w_2 = 1.5 & w_3 = 2.3 \\ w_4 = 3 & w_5 = 1 & w_6 = 1 \end{bmatrix}$$

$$\hat{y} = h_3$$

$$h_3 = w_5 \cdot o_1 + w_6 \cdot o_2$$

$$o_1 = \sigma(h_1) = \sigma(w_1 \cdot x_1 + w_2 \cdot x_2)$$

$$o_2 = \sigma(h_2) = \sigma(w_3 \cdot x_1 + w_4 \cdot x_2)$$

$$L = (y - \hat{y})^2.$$

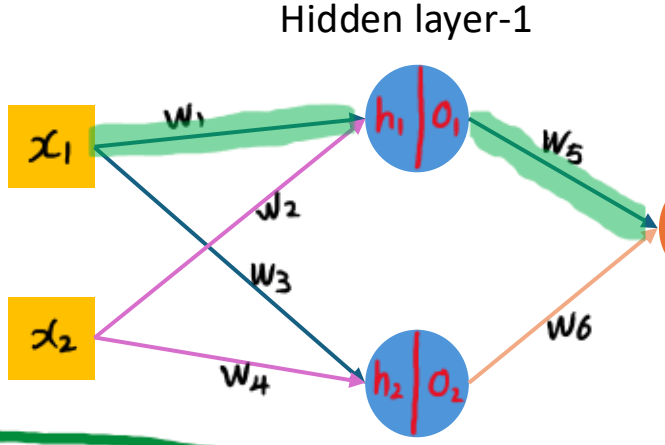
$$\frac{\partial L}{\partial w_5} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial h_3} \cdot \frac{\partial h_3}{\partial w_5}$$

$$= -2 \cdot (y - \hat{y}) \times 1 \times o_1$$

$$= -2 \times (4 - 5.05) \times 1 \times 1.25$$

$$= 2.625$$

Example (3)



$$\sigma(x) = \text{ReLU}(x) = \begin{cases} x & : \text{if } x \geq 0 \\ 0 & : \text{otherwise} \end{cases}$$

$\langle x = [1,\ 0.5]^T,\quad y = 4 \rangle$

$$\begin{bmatrix} w_1 = 0.5, & w_2 = 1.5 & w_3 = 2.3 \\ w_4 = 3 & w_5 = 1 & w_6 = 1 \end{bmatrix}$$

$$\hat{y} = h_3$$

$$h_3 = w_5 \cdot o_1 + w_6 \cdot o_2$$

$$o_1 = \sigma(h_1) = \sigma(w_1 \cdot x_1 + w_2 \cdot x_2)$$

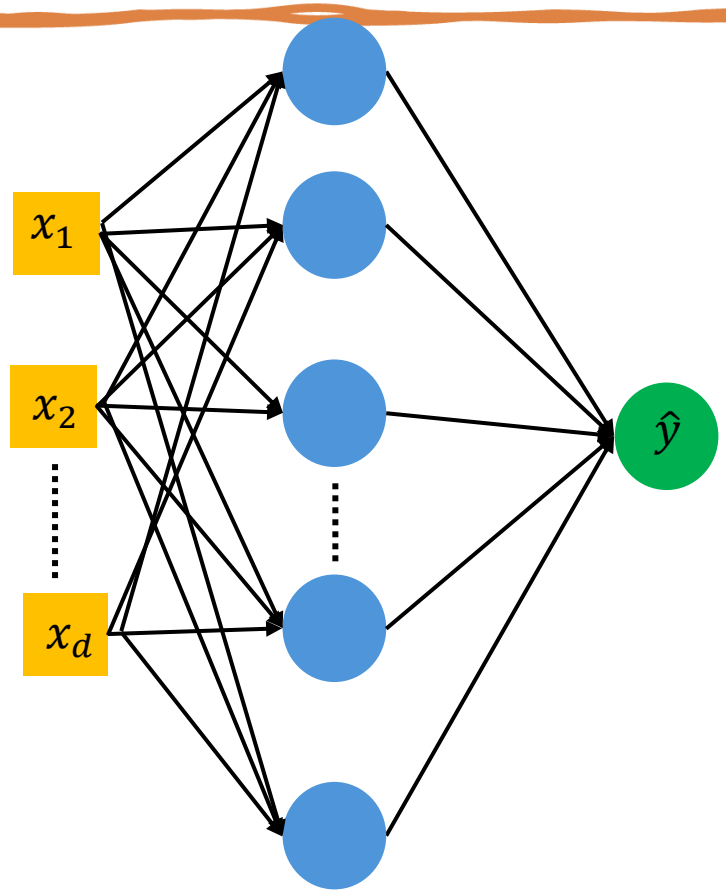
$$o_2 = \sigma(h_2) = \sigma(w_3 \cdot x_1 + w_4 \cdot x_2)$$

$$L = (y - \hat{y})^2.$$

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial h_3} \cdot \frac{\partial h_3}{\partial o_1} \cdot \frac{\partial o_1}{\partial h_1} \cdot \frac{\partial h_1}{\partial w_1}$$

$$= \ldots$$

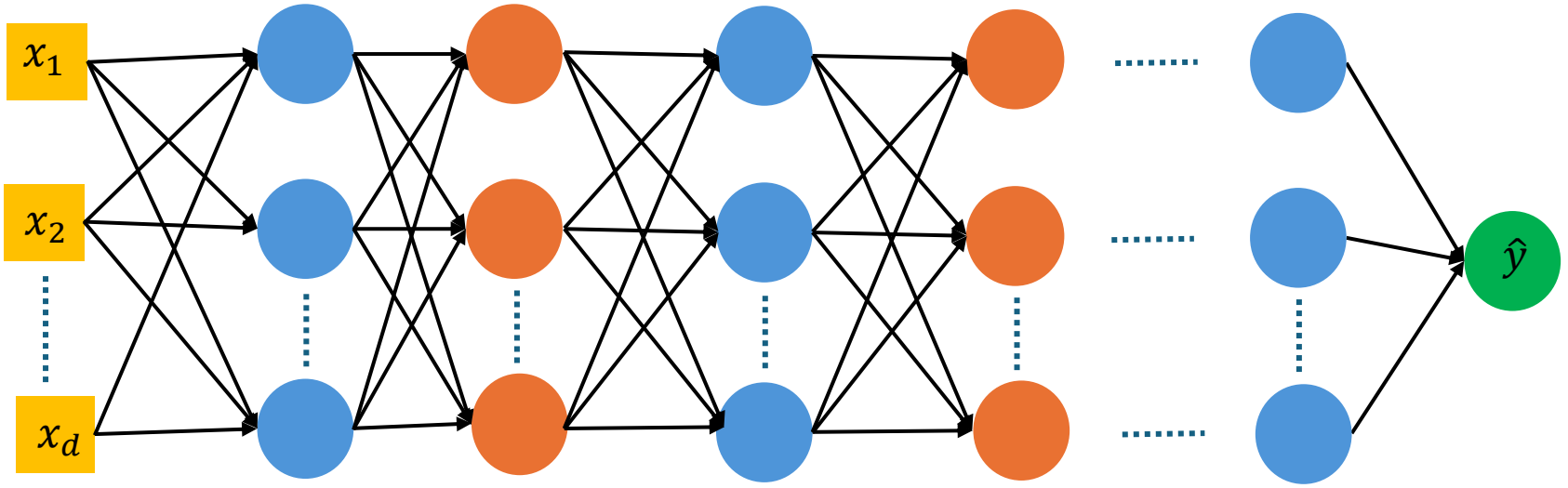
Architecture: width



Larger hidden layer

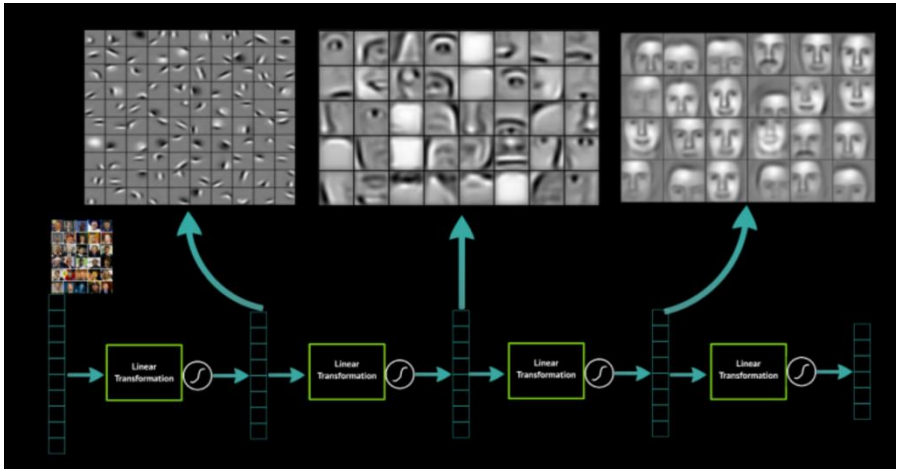
Architecture: depth

More hidden layers



Architecture: depth & width

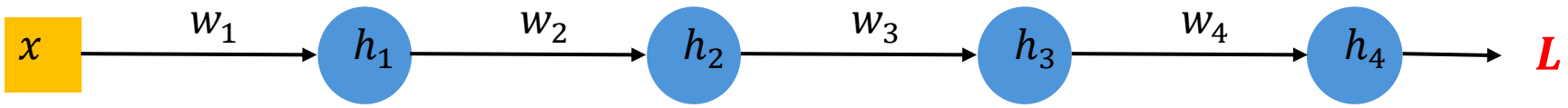
- Width and depth are both important and should be carefully tuned together for the best performance of neural networks
  - larger layers mean the network can "remember" more features (or the loss of information in the forwarding pass).
  - more layers mean it can "remember" more complex features (the abstraction level)



https://www.datarobot.com/blog/a-primer-on-deep-learning/

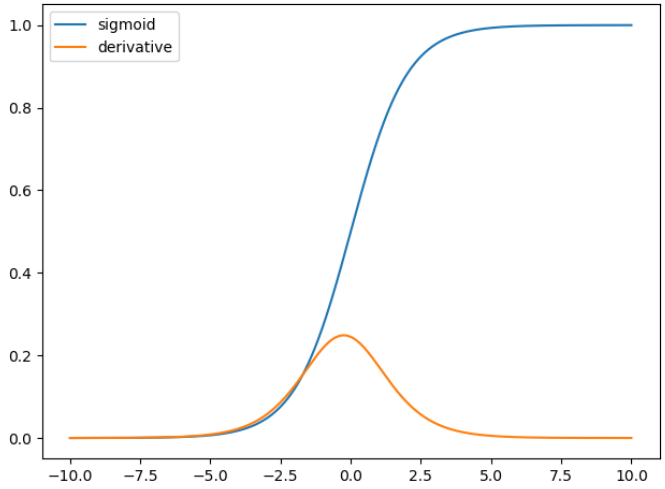
Lu Z, Pu H, Wang F, Hu Z, Wang L. The expressive power of neural networks: A view from the width. Advances in neural information processing systems. [NeurIPS 2017]

Vanishing Gradients (1)

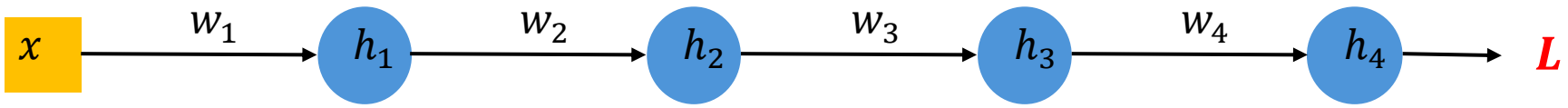


Activation function: $\sigma(x) = \frac{1}{1+e^{-x}}$

And Derivative function: $\sigma'(x) = \sigma(x)\big(1 - \sigma(x)\big)$



Vanishing Gradients (2)

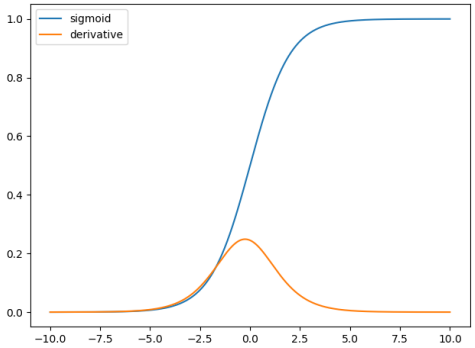


$$o_i = \sigma(w_i o_{i-1})$$

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial O_4} \times \frac{\partial O_4}{\partial O_3} \times \frac{\partial O_3}{\partial O_2} \times \frac{\partial O_2}{\partial O_1} \frac{\partial O_1}{\partial w_1}$$

$$= \frac{\partial L}{\partial O_4} \times \sigma'(o_3) \times w_4 \times \sigma'(o_2) \times w_3 \times \sigma'(o_1) \times w_2$$

$$< \frac{1}{4} \qquad < \frac{1}{4} \qquad < \frac{1}{4}$$



Exploding Gradients

- Gradient can grow exponentially with the distance from the output layer. In this case, the weights and biases will become exponentially large, resulting in a failure to learn.

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial O_4} \times \frac{\partial O_4}{\partial O_3} \times \frac{\partial O_3}{\partial O_2} \times \frac{\partial O_2}{\partial O_1} \frac{\partial O_1}{\partial w_1}$$
$$= \frac{\partial L}{\partial O_4} \times \underbrace{\sigma'(o_3) \times w_4}_{>1} \times \underbrace{\sigma'(o_2) \times w_3}_{>1} \times \underbrace{\sigma'(o_1) \times w_2}_{>1}$$

- The gradient explosion problem can be mitigated to some extent by gradient thresholding, that is, by resetting the value if it exceeds an upper bound.

How to identify a vanishing or exploding gradients problem?

- Vanishing
  - Large changes are observed in parameters of later layers, whereas parameters of earlier layers change slightly or stay unchanged
  - In some cases, weights of earlier layers can become 0 as the training goes
  - The model learns slowly and often times, training stops after a few iterations
  - Model performance is poor

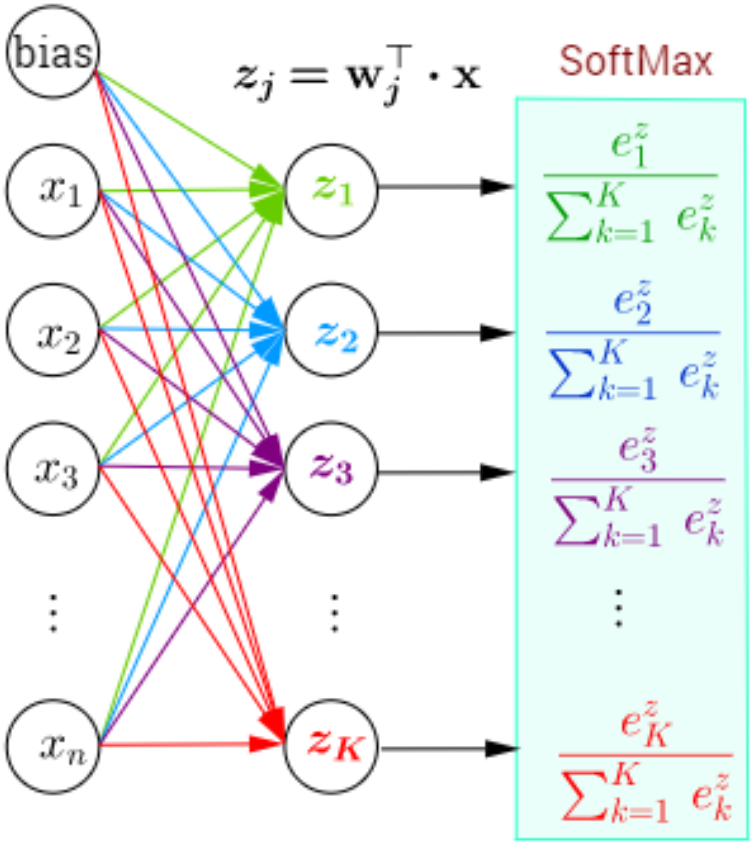
- Exploding:
  - Contrary to the vanishing scenario, exploding gradients shows itself as unstable, large parameter changes from batch/iteration to batch/iteration
  - Model weights can become NaN very quickly
  - Model loss also goes to NaN

Practices to fix the problems

- Rectified linear unit (ReLU)
- Batch normalization
- Weight initialization
- Gradient Clipping
- Skip connections
- Reduced network depth

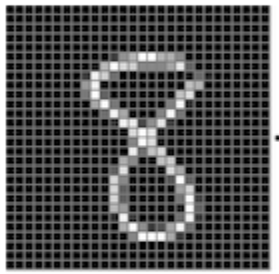
Multi-Class Classification with NN and SoftMax Function

$$\mathbf{z} = \begin{bmatrix} z_1 \\ z_2 \\ z_3 \\ \vdots \\ z_K \end{bmatrix} = \begin{bmatrix} \mathbf{w}_1^\top \\ \mathbf{w}_2^\top \\ \mathbf{w}_3^\top \\ \vdots \\ \mathbf{w}_K^\top \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_n \end{bmatrix}$$



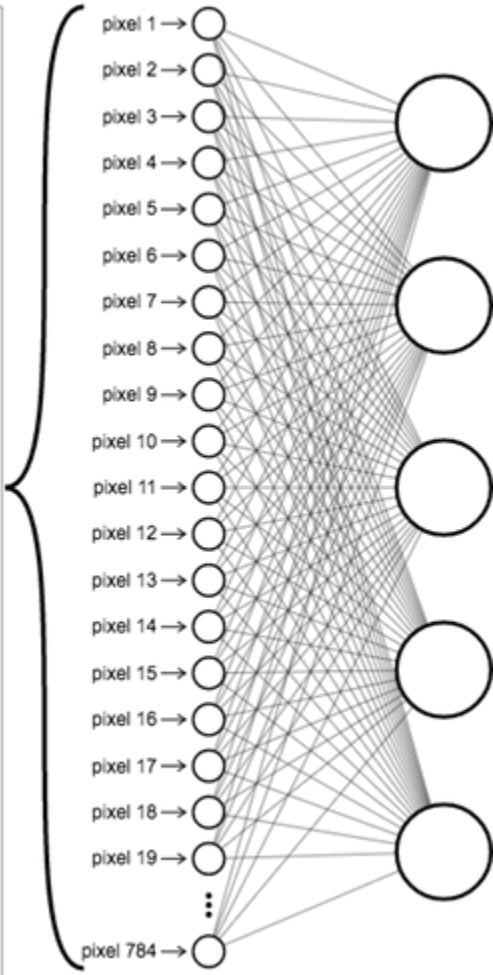


Example:
Single digit classification with neural network



28 x 28
784 pixels

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	4	62	146	182	254	254	181	176	139	15	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	34	186	253	217	208	136	136	136	166	232	99	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	61	242	208	111	3	0	0	0	0	0	18	32	107	43	0	0	0	0	0	0
0	0	0	0	0	0	0	0	156	242	23	0	0	0	0	0	0	0	13	191	181	6	0	0	0	0	0	0
0	0	0	0	0	0	0	0	121	255	98	3	0	0	0	0	0	8	194	225	12	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	169	253	120	3	0	0	0	0	128	247	51	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	3	111	244	169	19	0	14	131	249	117	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	59	241	235	72	142	229	66	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	25	218	254	231	36	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	133	253	221	33	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	19	237	111	196	217	19	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	174	138	0	23	193	204	18	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	96	224	0	0	0	25	218	169	3	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	215	138	0	0	0	0	86	253	99	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	215	97	0	0	0	0	3	162	214	11	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	215	97	0	0	0	0	0	118	253	68	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	185	157	0	0	0	0	0	40	254	98	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	50	244	61	0	0	0	0	112	244	58	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	174	251	142	59	83	167	244	111	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	6	133	253	253	253	169	61	3	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0



0-255: gray scale

Summary

- Multilayer ANN are universal approximators but could suffer from overfitting if the network is too large
- Naturally represents a hierarchy of features at multiple levels of abstractions
- Gradient descent may converge to local minimum
- Model building is computationally intensive, but testing is fast
- Can handle redundant and irrelevant attributes because weights are automatically learnt for all attributes
- Sensitive to noise in training data
  - This issue can be addressed by incorporating model complexity
- In the loss function
  - Difficult to handle missing attributes

Summary (Cont'd)

- Training **deep** neural networks (more than 5-10 layers) could only be possible in recent times with:
  - Faster computing resources (GPU)
  - Larger labeled training sets
- Algorithmic Improvements in Deep Learning
  - Responsive activation functions (e.g., RELU)
  - Regularization (e.g., Dropout)
  - Supervised pre-training
  - Unsupervised pre-training (auto-encoders)
  - Self-Supervised learning
- Specialized ANN Architectures:
  - Convolutional Neural Networks (for image data)
  - Recurrent Neural Networks (for sequence data)
  - Residual Networks (with skip connections)
  - Transformer
- Generative Models: Generative Adversarial Networks, Diffusion Model;